॥वसुधैव कुटुम्बकम्॥

# SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

**Symbiosis International (Deemed University)**

(Established under section 3 of the UGC Act, 1956)

**Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC**

**Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)**

## Assignment No. 14

| | |
|---|---|
| **Subject:** | DevOps Lab |
| **TCODE:** | TE7950 |
| **Name of Student** | **Yashmit Kotekar** |
| **PRN No.** | **23070122236** |
| **Branch** | CSE, Batch (2023-27), L4 |
| **Academic Year & Semester** | 25 – 26, Sem VII |
| **Date of Performance** | 23rd July, 2026 |
| **Title of Assignment:** | CI/CD: Jenkins Pipeline building Docker Image |
| **Theory:** | A Jenkins Pipeline automates the Continuous Integration and Continuous Deployment (CI/CD) process by executing a series of predefined stages such as source code checkout, build, test, and deployment. In this experiment, a Docker image was built automatically using Jenkins after fetching the application source code from a GitHub repository. The docker build command created a versioned Docker image using the Jenkins build number as the image tag, ensuring unique image versions for every pipeline execution. This integration of Jenkins and Docker enables consistent, repeatable, and automated application builds, reducing manual effort and improving software delivery efficiency.<br><br>**Commands Used**<br>• docker build -t my-jenkins-docker .<br>• docker run -d --name myjenkins --restart unless-stopped -u root -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock my-jenkins-docker<br>• docker exec -it myjenkins docker --version<br>• docker exec -it myjenkins rm -rf /var/jenkins_home/workspace/Declarative-Pipeline-Demo<br>• git add .<br>• git commit -m "Add Dockerfile for Jenkins pipeline"<br>• git push origin main<br>• docker images |

**Implementation:**

**Step 1: Access and Edit Your Pipeline Configuration**

Accessed Jenkins Pipeline Configuration — Opened the existing Declarative Jenkins Pipeline and navigated to the Pipeline configuration section to modify the pipeline script.

**Step 2: Add the Build Docker Image Stage**

Added Docker Build Stage — Modified the existing Declarative Jenkins Pipeline by adding a new stage to build the Docker image using the docker build command with the Jenkins build number as the image tag.

**Step 3: Run the Pipeline**

Executed Jenkins Pipeline — Successfully executed the Jenkins Declarative Pipeline containing the Docker image build stage. The pipeline completed successfully, and the Docker image was built and tagged without errors.

**Step 4: Verify the Docker Image**

Verified Docker Image — Verified that the Docker image (my-flask-app:8) was successfully built by the Jenkins Pipeline and stored in the local Docker image repository using the docker images command.

Jenkinsfile U

TW2 > Task3-Jenkins-Terraform > Jenkinsfile

```
pipeline {
    agent any

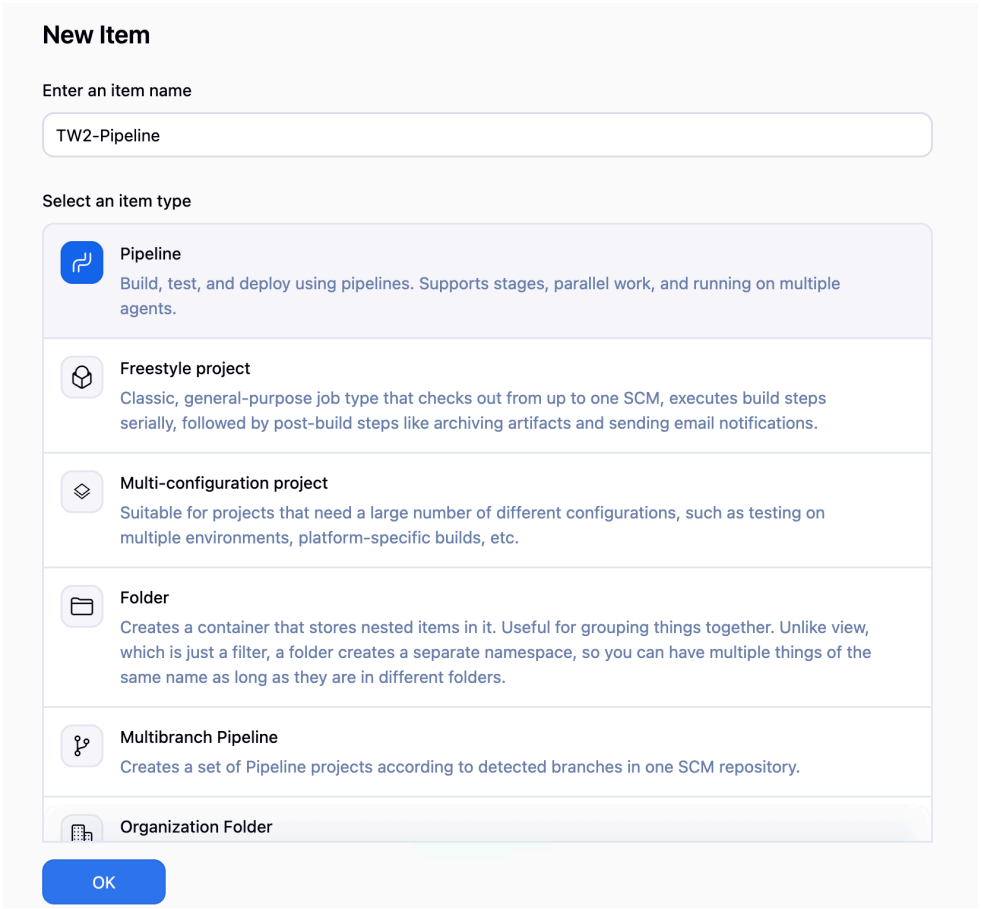
    stages {

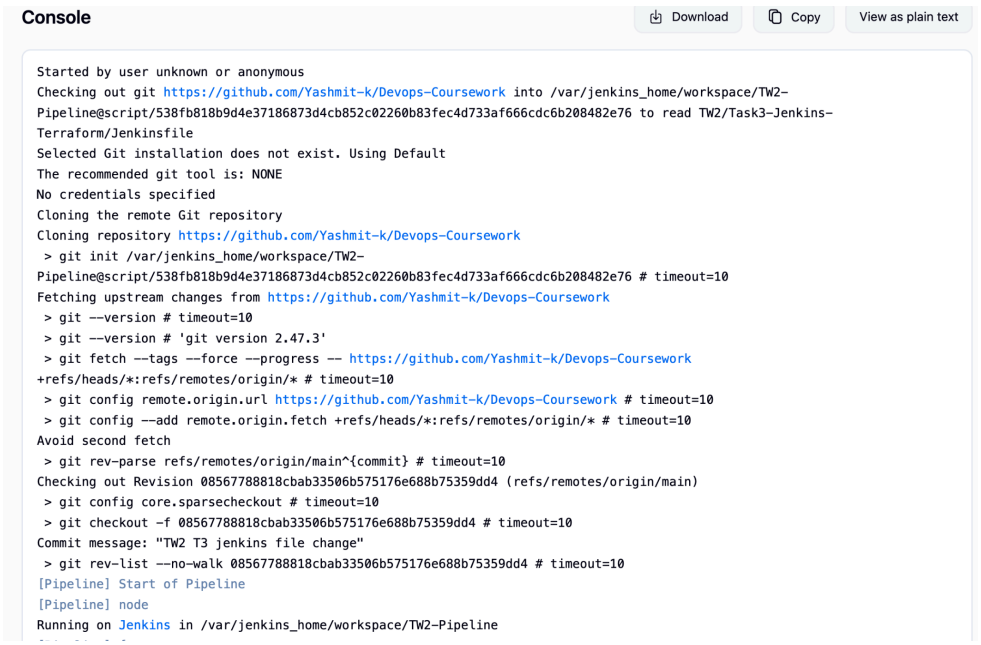
        stage('Checkout') {
            steps {
                git 'git@github.com:Yashmit-k/Devops-Coursework.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t flask-hello ./TW2/Task1-Docker-Compose'
            }
        }

    }

    post {
        success {
            echo 'Pipeline completed successfully!'
        }
    }
}
```


New Item
Enter an item name
TW2-Pipeline
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
Organization Folder
OK


Console
Download
Copy
View as plain text
Started by user unknown or anonymous
Checking out git https://github.com/Yashmit-k/Devops-Coursework into /var/jenkins_home/workspace/TW2-Pipeline@script/538fb818b9d4e37186873d4cb852c02260b83fec4d733af666cdc6b208482e76 to read TW2/Task3-Jenkins-Terraform/Jenkinsfile
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/Yashmit-k/Devops-Coursework
> git init /var/jenkins_home/workspace/TW2-Pipeline@script/538fb818b9d4e37186873d4cb852c02260b83fec4d733af666cdc6b208482e76 # timeout=10
Fetching upstream changes from https://github.com/Yashmit-k/Devops-Coursework
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --tags --force --progress -- https://github.com/Yashmit-k/Devops-Coursework +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/Yashmit-k/Devops-Coursework # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 08567788818cbab33506b575176e688b75359dd4 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 08567788818cbab33506b575176e688b75359dd4 # timeout=10
Commit message: "TW2 T3 jenkins file change"
> git rev-list --no-walk 08567788818cbab33506b575176e688b75359dd4 # timeout=10
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/TW2-Pipeline

```
Running on Jenkins in /var/jenkins_home/workspace/TW2-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/Yashmit-k/Devops-Coursework
 > git init /var/jenkins_home/workspace/TW2-Pipeline # timeout=10
Fetching upstream changes from https://github.com/Yashmit-k/Devops-Coursework
 > git --version # timeout=10
 > git --version # 'git version 2.47.3'
 > git fetch --tags --force --progress -- https://github.com/Yashmit-k/Devops-Coursework
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/Yashmit-k/Devops-Coursework # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 08567788818cbab33506b575176e688b75359dd4 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 08567788818cbab33506b575176e688b75359dd4 # timeout=10
Commit message: "TW2 T3 jenkins file change"
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build Docker Image)
[Pipeline] sh
+ docker build -t flask-hello ./TW2/Task1-Docker-Compose
#0 building with "default" instance using docker driver
```

```
#7 [2/5] WORKDIR /app
#7 CACHED

#8 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED

#9 [5/5] COPY . .
#9 DONE 0.0s

#10 exporting to image
#10 exporting layers 0.0s done
#10 exporting manifest sha256:1a18e6e4ff7d96836c7182521dcdac8ff67e9cd9772535a7e26a292767a8dfc7 done
#10 exporting config sha256:9701e1c160bb5c1e1426ae5a282d70aacc7ecc3460dab3715bb236ccf23015dc done
#10 exporting attestation manifest sha256:5d4052d39f9f35cfa333db96e5c82e33931f4fef85723345db77b37634dc2f51 done
#10 exporting manifest list sha256:e32f915ef709e763bddb4ccd5e824dc015367c5edf7daa910014e0e90b676fa5 done
#10 naming to docker.io/library/flask-hello:latest done
#10 unpacking to docker.io/library/flask-hello:latest done
#10 DONE 0.0s
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Subject Faculty Name: Prof. Arijit Dutta